FOR THE TALKER:

IMPORTING THE LIBRARIES:

```
from example_interfaces.msg import String
```

This imports ROS 2 message type string , this is because ROS 2 doesn't send raw python objects via a topic it sends ROS messages which have a predefined structure

```
import rclpy
```

This imports ROS 2 python client library allowing you to create things like nodes etc using python

```
from rclpy.executors import ExternalShutdownException
```

This imports an exception so that the program (in this case talker.py) exits cleanly without fails or error message should in case ROS 2 were to stop externally

```
from rclpy.node import Node
```

This imports the node class

THE BODY:

```
class Talker(Node):
```

This creates the talker class and it inherits from the parent Node making it possible to gain all the functionality in the parent

```
    def __init__(self):
        super().__init__('talker')
        self.i = 0
        self.pub = self.create_publisher(String, 'chatter', 10)
        timer_period = 1.0
        self.tmr = self.create_timer(timer_period, self.timer_callback)
```

- Def __init__(self) in python is called a constructor and it runs every time a talker object is created and is used to initialize starting values of the object
- Super().__init__('talker'): this loads the parent class first (in this case Node) making ROS 2 functionality available to the talker class
- Self.pub creates the publisher that would send the message via the topic 'chatter' with a queue of 10.
- Self.I is a counter variable and is used to keep track of how many messages we are printing on the terminal. It is initialized to 0
- Timer_period is set to 1 sec .
- Self.tmr is a timer that expires every second and once it does ROS2 calls the timer_callback function that which creates and publish a message.

```
    def timer_callback(self):
        msg = String()
        msg.data = 'Hello World: {0}'.format(self.i)
        self.i += 1
        self.get_logger().info('Publishing: "{0}"'.format(msg.data))
        self.pub.publish(msg)
```

The timer_callback function creates and publishes the message via the topic channel. First the message is defined by calling the string object. We define the message data9 a formatted string that is replaced by self .I after every update). the latter part of the code block publishes the message.

```
def main(args=None):
    try:
        with rclpy.init(args=args):
            node = Talker()
```

```
            rclpy.spin(node)
    except (KeyboardInterrupt, ExternalShutdownException):
        pass
```

This part of the codes is what actually runs the code and handles error(read exception handling). the try part runs the code where we define the object , rclpy.spin(node) keeps the code running till an event occurs. While the except portion has 2 event it runs `ExternalShutdownException` which was explained earlier and KeyboardInterrupt an event that occurs when u use certain keys like ctrl + c to terminate the code sequence.

```
if __name__ == '__main__':
    main()
```

LISTENER:

```
import rclpy
from rclpy.executors import ExternalShutdownException
from rclpy.node import Node
```

same libraries imported as in the talker

```
class Listener(Node):
   def __init__(self):
        super().__init__('listener')
        self.sub = self.create_subscription(String, 'chatter', self.chatter_callback, 10)
```

Self.sub does 4 things:

- Specifies the type of message the subscriber expects
- Specifies the topic “chatter” in this case
- Specifies the function `self.chatter_callback` to run whenever a message arrives
- Specifies the queue depth. Up to 10 messages can be buffered if the listener cant process them immediately

```
    def chatter_callback(self, msg):
        self.get_logger().info('I heard: [%s]' % msg.data)
```

The chatter _callback function basically displays the message it received from the talker.

```
def main(args=None):
    try:
        with rclpy.init(args=args):
            node = Listener()
            rclpy.spin(node)
    except (KeyboardInterrupt, ExternalShutdownException):
        pass
```

Same as above

```
if __name__ == '__main__':
    main()
```